SOMAIYA VIDYAVIHAR UNIVERSITY
K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

Somaiya TRUST

Experiment No-3
Title: Preparation of Software Requirements Specifications (SRS).

Batch: **Roll No.:** **Experiment No: 3**

Aim: Preparation of Software Requirements Specifications (SRS).

Resources needed: LaTeX Editor, Internet Browser

Theory

Requirements specify how the target system should behave. It specifies what to do, but not how to do. Requirements engineering refers to the process of understanding what a customer expects from the system to be developed, and to document them in a standard and easily readable and understandable format. This documentation will serve as reference for the subsequent design, implementation and verification of the system.

It is necessary and important that before we start planning, design and implementation of the software system for our client, we are clear about its requirements. If we don't have a clear vision of what is to be developed and what all features are expected, there would be serious problems and customer dissatisfaction as well.

Characteristics of Requirements

Requirements gathered for any new system to be developed should exhibit the following three properties:

1. Unambiguity: There should not be any ambiguity what a system to be developed should do. For example, consider you are developing a web application for your client.

The client requires that enough number of people should be able to access the application simultaneously. What's the "enough number of people"? That could mean 10 to you but, perhaps, 100 to the client. There's an ambiguity.

2. Consistency: To illustrate this, consider the automation of a nuclear plant. Suppose one of the clients say that it the radiation level inside the plant exceeds R1, all reactors should be shut down. However, another person from the client side suggests that the threshold radiation level should be R2. Thus, there is an inconsistency between the two end users regarding what they consider as threshold level of radiation.

3. Completeness: A particular requirement for a system should specify what the system should do and also what it should not. For example, consider software to be developed for ATM. If a customer enters an amount greater than the maximum permissible withdrawal amount, the ATM should display an error message, and it should not dispense any cash.

Categorization of Requirements

Based on the target audience or subject matter, requirements can be classified into different types, as stated below:

1. User requirements: They are written in natural language so that both customers can verify their requirements have been correctly identified

2. System requirements: They are written involving technical terms and/or specifications, and are meant for the development or testing teams

Requirements can be classified into two groups based on what they describe:

1. Functional requirements (FRs): These describe the functionality of a system -- how a system should react to a particular set of inputs and what should be the corresponding output.

2. Non-functional requirements (NFRs): They are not directly related what functionalities are expected from the system. However, NFRs could typically define how the system should behave under certain situations. For example, a NFR could say that the system should work with 128MB RAM. Under such condition, a NFR could be more critical than a FR.

IEEE SRS document (Relevant IEEE standards: IEEE-830[5])

The SRS is a specification of the requirements for the software "product" you will produce in your project. The basic issues to be addressed are:

a) Functionality. What is the software supposed to do?

b) External interfaces. How should the software interact with people, the operating system, hardware, networks, and other software?

c) Performance. What are the requirements for speed, availability, response time, recovery time of various software functions, etc.?

d) Quality Attributes. What are the requirements for portability, correctness, maintainability, security, etc?

e) Design constraints imposed on an implementation. Is there a requirement for a particular programming language? Are there resource limits (such as disk or memory size)? Must it run on a particular operating system? Must it inter-operate with particular web browsers? etc.

The SRS contains requirements, and not your design solutions; it is the "what" and not the "how" of your project. The information is collected from the project client. In a good SRS, the requirements should be: Correct, Unambiguous, Complete, Consistent, and Ranked for importance, Verifiable, and Traceable.

1 INTRODUCTION

1.1 Product Overview

Clearly define the purpose of the software, the environment it will run in, and who will be using it (in terms of their educational level, experience, and technical expertise). Do not go into detail, but outline the general requirements in a way that provides the reasons why specific requirements are later specified in the SRS.

2 SPECIFIC REQUIREMENTS

This section of the SRS should contain all of the software requirements to a level of detail sufficient to:

1. enable the SRS to be checked by the originator of the original system requirements (usually the project supervisor),
2. enable designers to design a system to satisfy those requirements, and
3. enable testers to test that the system satisfies those requirements.

Throughout this section, every stated requirement should be externally perceivable by users, or other external systems. This ensures that all features are testable.

2.1 External Interface Requirements

2.1.1 User Interfaces

This should specify the following:

a) The characteristics of the user interface. Include the characteristics necessary to accomplish the software requirements (for example: required screen formats, page or window layouts, content of any reports or menus, or availability of programmable function keys).

b) All the aspects of optimizing the interface with the person who must use the system. This may simply comprise a list of do's and don'ts on how the system will appear to the user. Like all others, these requirements should be verifiable.

2.1.2 Hardware Interfaces

This should specify the characteristics of each interface between the software and the hardware components of the system. This includes configuration characteristics (number of ports, instruction sets, etc.). It also covers such matters as what devices are to be supported, how they are to be supported, and protocols. Only use this section if your project requires specific hardware to be used – do not, for example, specify the hardware if there is a requirement to establish a network connection.

2.1.3 Software Interfaces

This should specify the use of other required software which your software must interface with (for example: a database system, an operating system, or a mathematical package).

For each required software product, the following should be provided:

- Name and Version number.

For each interface, the following should be provided:

- Discussion of the purpose of the interfacing software as related to this software product.
- Definition of the interface in terms of message content and format. It is not necessary to detail any well-documented interface, but a reference to the document defining the interface is required.

2.1.4 Communications Protocols

This should specify the various interfaces to communications such as local network protocols, etc. Make reference to any well-defined protocols, specifying exactly which parts or options of that protocol the software needs to support.

2.2 Software Product Features

This section should consist of a numbered list of required features.

Typically, all of the requirements that relate to a software product are not equally important. The importance of each feature should be indicated using one of the following terms: essential, important, or desirable.

Each feature requirement should include:

? ? a description of every input (stimulus) into the system,
? ? a description of every output (response) from the system,
? ? a description of every state change within the system,
? ? a description of all the functions performed by the system in response to an input or in support of an output.

Functional requirements should define the actions that must take place in the software in accepting and processing the inputs and in processing and generating the outputs. These are generally listed as "shall" statements starting with "The system shall". These include
a) Validity checks on the inputs
b) Exact sequence of operations
c) Responses to abnormal situations, including
1) Overflow
2) Communication facilities
3) Error handling and recovery
d) Effect of parameters
e) Relationship of outputs to inputs, including
1) Input/output sequences
2) Formulas for input to output conversion

It may be appropriate to partition the functional requirements into sub functions or sub processes. This does not imply that the software design will also be partitioned that way.

2.3 Software System Attributes

2.3.1 Reliability

Specify the required reliability of the final software system.
This is particular important for applications such as embedded software. It can be quantified using MTTF (Mean Time To Failure) measurements.

2.3.2 Availability

Specify the required availability of the final software system: define requirements such as check pointing, recovery, and restart.

2.3.3 Security

Specify the factors that protect the software from accidental or malicious access, use, modification, destruction, or disclosure.
Specific requirements in this area could include the need to
a) Utilize certain crypto graphical techniques;
b) Keep specific log or history data sets;
c) Assign certain functions to different modules;
d) Restrict communications between some areas of the program;
e) Check data integrity for critical variables.

2.3.4 Maintainability

This should specify attributes of the software that relate to the ease of maintenance of the software itself. Specify any requirements for certain modularity, interfaces, complexity, etc that make the software easier to maintain.

Requirements should not be placed here just because they are thought to be good design practices.

2.3.5 Portability

This should specify attributes of software that relate to the ease of porting the software to other host machines and/or operating systems. This may include the following:

a) Percentage of code that is host dependent;
b) Use of a proven portable language;
c) Use of a particular compiler or language subset;
d) Use of a particular operating system.

2.3.6 Performance

This subsection should specify both the static and the dynamic numerical requirements placed on the software or on human interaction with the software as a whole.

Static numerical requirements may include, for example, the following: minimum number of simultaneous users, minimum data storage, etc.

Dynamic numerical requirements may include, for example, the required number of transactions per second for both normal and peak workload conditions, etc.

All of these requirements should be stated in measurable terms.

2.4 Database Requirements

This should specify the logical requirements for any information that is to be placed into a database. This may include the following:

a) Types of information used by various functions;
b) Accessing capabilities;
c) Data entities and their relationships;
d) Integrity constraints.

Activities:

Prepare SRS document for chosen problem definition in LaTeX.

Results:

1. INTRODUCTION

1.1 Product Overview

The **Predictive Modeling of the RainMumbai Index Using Meteorological Data** project aims to develop a Machine Learning-based web application for predicting the RainMumbai Index using historical and current meteorological data. The primary objective of the system is to analyze weather-related parameters and generate predictions regarding the expected movement and deviation of the RainMumbai Index.

The system will obtain meteorological data from suitable datasets and external APIs. The collected data will be preprocessed and provided to the trained Machine Learning model, which will generate RainMumbai Index predictions along with associated confidence or probability measures.

Users will interact with the system through a web-based dashboard. The dashboard will display the current prediction, expected deviation of the index, relevant meteorological parameters, prediction confidence, historical predictions, and past model performance. The system will also provide profitability statistics based on historical predictions and defined decision rules, allowing users to evaluate the historical performance of the model. Such statistics will represent past model performance and will not guarantee future profitability.

The application will include user registration, login, logout, account management, and subscription functionality. Registered users with an active subscription will be able to access prediction and analytical features provided by the platform.

The system will be designed to operate on standard computing infrastructure and will not require specialized hardware. The development and model-training environment should have a modern multi-core processor and approximately 16 GB of memory for efficient data processing and Machine Learning operations.

The intended users include individuals interested in rainfall analysis, weather-based prediction, quantitative analysis, and the RainMumbai Index. The interface will be designed so that users can understand the predictions and model statistics without requiring advanced knowledge of Machine Learning or meteorology.

2. SPECIFIC REQUIREMENTS

The system shall provide the necessary functionality for meteorological data acquisition, data preprocessing, Machine Learning-based RainMumbai Index prediction, prediction confidence estimation, historical performance analysis, user management, subscription management, and visualization through a web-based dashboard.

The requirements specified in this section are intended to ensure that the system can be designed, implemented, evaluated, and tested according to the objectives of the project.

2.1 External Interface Requirements

2.1.1 User Interfaces

The primary user interface shall be a responsive web application accessible through commonly used modern web browsers.

The system shall provide the following major interfaces:

1. **Registration Page:** The system shall allow new users to create an account by providing required information such as name, email address, contact information, and password.
2. **Login and Logout Interface:** Registered users shall be able to securely log in and log out of the application.
3. **User Dashboard:** The dashboard shall provide a summarized view of RainMumbai Index predictions, prediction confidence, relevant meteorological parameters, historical predictions, and model performance.
4. **Prediction Interface:** The system shall display the predicted RainMumbai Index value or movement, expected deviation, and associated confidence/probability information.
5. **Historical Prediction Interface:** Users shall be able to view previous predictions generated by the model and compare them with observed outcomes where such data is available.
6. **Profitability Analysis Interface:** The system shall display historical profitability statistics calculated using model predictions and predefined decision rules. The interface shall clearly distinguish historical performance from future expected results.
7. **Subscription Interface:** Users shall be able to view their subscription status and access subscription-related functionality.

The interface shall use understandable labels, graphs, tables, indicators, and other visualization components. Important prediction information shall be clearly distinguishable from supporting statistics.

The system shall avoid presenting model predictions as guaranteed outcomes. Confidence values, historical accuracy, and profitability statistics shall be appropriately labelled to help users interpret the results.

2.1.2 Hardware Interfaces

The proposed system does not require any specialized hardware or dedicated physical devices. The development, model training, and testing environment should have the following minimum recommended configuration:

- Modern multi-core processor
- Approximately 16 GB RAM
- Sufficient storage for meteorological datasets, trained models, application files, and database records
- Standard network interface for Internet connectivity

Users shall be able to access the deployed web application using a normal desktop computer, laptop, tablet, or other compatible device with a modern web browser and Internet connection. GPU-based hardware may be used during model experimentation or training if required but shall not be mandatory for normal application usage.

2.1.3 Software Interfaces

The system shall interface with multiple software components required for meteorological data processing, Machine Learning prediction, web application functionality, and data storage.

Machine Learning Environment:

Python shall be used for data preprocessing, feature engineering, model development, model training, prediction, and evaluation. Appropriate data analysis and Machine Learning libraries may be used depending on the final model architecture.

Meteorological Data Interface:

The system shall obtain meteorological data from selected datasets and/or external APIs. Data obtained from these sources shall be transformed into the format required by the Machine Learning pipeline.

Database Interface:

A suitable database management system shall be used to store user information, prediction history, model outputs, subscription information, and other application-related data.

Web Application Interface:

The frontend web application shall communicate with the application backend to retrieve prediction results, historical statistics, user information, and other dashboard data.

Machine Learning Model Interface:

The backend shall provide the required meteorological parameters to the deployed ML model. The model shall return the predicted RainMumbai Index output and relevant prediction statistics to the application.

The exact versions of Python, Machine Learning libraries, database technology, frontend framework, and backend framework shall be documented after the final technology stack is selected.

2.1.4 Communications Protocols

The system does not require any specialized communication protocol.

Standard Internet communication protocols shall be sufficient for communication between users, the web application, backend services, database services, and external data APIs.

The deployed application shall use HTTPS for secure communication between the user's browser and the server. External meteorological APIs shall be accessed using the protocols and data formats supported by the selected data provider, such as HTTPS-based REST APIs with JSON or other supported formats.

2.2 Software Product Features

1. User Registration — Essential

The system shall allow a new user to register by providing the required account information.

The system shall validate mandatory fields and shall reject invalid or duplicate account information where applicable.

After successful registration, the system shall create a user account and store the required information securely.

2. User Authentication — Essential

The system shall provide secure login and logout functionality.

The system shall verify the user's authentication credentials before providing access to protected functionality.

The system shall prevent unauthorized users from accessing subscription-only prediction and analytical features.

3. Meteorological Data Acquisition — Essential

The system shall obtain the meteorological data required for RainMumbai Index prediction from selected datasets or external APIs.

The system shall validate incoming data before supplying it to the prediction pipeline.

The system shall appropriately handle missing, invalid, delayed, or unavailable data.

4. Data Preprocessing — Essential

The system shall preprocess meteorological data before providing it to the trained Machine Learning model.

Preprocessing may include handling missing values, data transformation, normalization, feature selection, feature engineering, and conversion of input data into the format expected by the model.

The same preprocessing logic used during model development shall be consistently applied during prediction.

5. RainMumbai Index Prediction — Essential

The system shall use a trained Machine Learning model to generate predictions for the RainMumbai Index.

The prediction process shall use the available meteorological parameters and engineered features as model inputs.

The system shall provide the predicted index value, expected movement, deviation, or other prediction output according to the final formulation of the RainMumbai Index prediction problem.

6. Prediction Confidence — Essential

The system shall provide an appropriate measure of model confidence or prediction uncertainty where supported by the selected Machine Learning methodology.

The dashboard shall display confidence/probability information in an understandable form.

The system shall not represent confidence values as guarantees of prediction correctness.

7. Meteorological Parameter Visualization — Important

The system shall display important meteorological parameters influencing or associated with the current prediction.

Users shall be able to examine relevant weather information alongside the model's predicted output.
Suitable charts, indicators, and statistical representations shall be used where appropriate.

8. Historical Prediction Tracking — Essential

The system shall maintain records of predictions generated by the ML model.
The system shall allow users to view past predictions and, where available, compare predicted values with actual observed outcomes.
Historical records shall contain sufficient information to evaluate the past performance of the model.

9. Prediction Performance Analysis — Essential

The system shall evaluate historical predictions using appropriate Machine Learning performance metrics.
The selected metrics shall depend on whether the final problem is implemented as a regression, classification, probabilistic forecasting, or combined prediction problem.
The results shall be presented to users through the dashboard in an understandable format.

10. Profitability Analysis — Important

The system shall calculate historical profitability based on model predictions and clearly defined decision rules.
The system shall allow users to view historical profit/loss information and aggregate performance statistics.
Profitability calculations shall be based on recorded historical information and shall not be represented as a guarantee of future financial returns.

11. User Dashboard — Essential

The system shall provide a centralized dashboard containing key information including:

- Current RainMumbai Index prediction
- Expected deviation or movement
- Prediction confidence/probability
- Relevant meteorological parameters
- Historical predictions
- Model performance
- Historical profitability
- User account and subscription status

The dashboard shall present information using suitable charts, tables, cards, and statistical indicators.

12. Subscription Management — Important

The system shall support subscription-based access to the prediction platform.
The system shall maintain the subscription status of each user and determine whether the user is authorized to access subscription-only functionality.
Where an external payment service is integrated, sensitive payment credentials shall be processed according to the requirements of the selected payment provider and shall not be stored directly by the application unless explicitly required and appropriately secured.

13. Model Updating — Essential

The Machine Learning model shall be evaluated and, where appropriate, retrained or updated approximately once every month using newly available meteorological and outcome data.

Before deployment, an updated model shall be evaluated against defined performance criteria.

The system shall maintain model version information so that predictions can be associated with the model version that generated them.

14. Error Handling — Essential

The system shall detect and appropriately respond to errors including invalid user input, unavailable meteorological data, API failures, authentication failures, database errors, and model prediction failures.

The system shall provide understandable error messages to users without exposing sensitive internal system information.

2.3 Software System Attributes

2.3.1 Reliability

The system shall consistently process valid meteorological inputs and generate prediction outputs using the deployed model.

Input validation and exception handling shall be implemented to prevent invalid or incomplete data from causing uncontrolled system failures.

The prediction pipeline shall produce reproducible results when provided with identical input data and the same model version, subject to the characteristics of the selected ML algorithm.

Historical predictions shall be stored so that model outputs can subsequently be verified and evaluated.

2.3.2 Availability

The deployed web application should be available to registered users whenever prediction services are operational.

Temporary failures of external meteorological data providers shall be handled without corrupting existing system data.

The system shall support recovery and restart of application services without loss of previously stored user and prediction information.

Scheduled maintenance, including monthly model updates, should be performed with minimum disruption to users.

2.3.3 Security

The system shall protect user information, model files, prediction records, and subscription information from unauthorized access.

User passwords shall not be stored as plain text and shall be protected using appropriate password hashing techniques.

Communication between users and the deployed web application shall use HTTPS.

Access to protected functionality shall require user authentication and appropriate authorization.

Sensitive user information shall only be accessible to authorized components and users.

The trained Machine Learning model, source code, API credentials, database credentials, and other confidential system resources shall not be directly exposed through the user interface.
The system shall maintain appropriate logs for authentication events, prediction requests, system errors, and other important operational activities.

2.3.4 Maintainability

The software shall follow a modular structure so that the web interface, backend services, database, data preprocessing pipeline, and Machine Learning model can be maintained independently.
The Machine Learning pipeline shall support periodic evaluation and updating as additional meteorological data becomes available.
The ML model shall be evaluated approximately every month, and retraining shall be performed when new data or model performance indicates that an update is required.
Model versions and relevant performance information shall be maintained to enable comparison between deployed models.
Changes to one major component should require minimal changes to unrelated components of the system.

2.3.5 Portability

The system shall be developed using widely supported programming languages, frameworks, and libraries.
The Machine Learning components shall primarily use Python to provide compatibility with commonly used operating systems and computing environments.
The web application shall be accessible through modern web browsers without requiring specialized client-side software.
The software should be deployable on local servers or suitable cloud infrastructure with minimal configuration changes.

2.3.6 Performance

The system shall provide prediction results within a reasonable response time after receiving valid meteorological inputs.
For normal operating conditions, the target response time for an individual prediction request should be less than **5 seconds**, excluding delays caused by unavailable or slow external data providers.
Normal dashboard pages should load within approximately **5 seconds** under expected network and server conditions.
The system shall be capable of storing historical meteorological observations, user records, predictions, and model performance data without significant degradation in normal application performance.
Model training and monthly retraining may require longer processing times and shall be treated separately from real-time user prediction requests.

2.4 Database Requirements

The system shall use a database to store information required for user management, prediction tracking, performance analysis, subscription management, and operation of the application.

The major categories of information stored by the system shall include:

User Data:

The database shall maintain information such as User ID, name, email address, contact information, authentication information, registration date, and account status.

Subscription Data:

The database shall maintain subscription information including subscription type, activation date, expiry date, payment status, and relevant transaction references. Sensitive payment information shall be handled according to the requirements of the selected payment service.

Meteorological Data:

The system may store meteorological observations required for model development, validation, and prediction. This may include rainfall, temperature, humidity, atmospheric pressure, wind-related variables, timestamps, location information, and other selected weather parameters.

Prediction Data:

Each prediction record shall contain the predicted RainMumbai Index output, prediction timestamp, relevant confidence or uncertainty information, model version, and other information required for historical evaluation.

Actual Outcome Data:

Where available, actual observed RainMumbai Index values or corresponding rainfall outcomes shall be stored to enable comparison between predictions and observed results.

Performance Data:

The database shall maintain information required to calculate model accuracy, historical prediction performance, and other evaluation metrics.

Profitability Data:

Where profitability analysis is implemented, the system shall store or derive information required to calculate historical profit/loss and aggregate profitability statistics according to predefined decision rules.

Model Information:

The database shall maintain metadata about deployed Machine Learning models, including model version, training date, evaluation results, and deployment status.

The database shall maintain relationships between users, subscriptions, prediction records, model versions, and relevant performance records.

Unique identifiers shall be assigned to important entities such as users, predictions, subscriptions, and model versions. Appropriate integrity constraints shall be applied to prevent duplicate, inconsistent, or invalid records.

Access to database information shall be restricted according to the user's role and authorization level. Sensitive information shall not be exposed directly through the client-side web application.

Questions:

1. Explain various steps involved in Requirement Engineering.

Requirement Engineering is the process of identifying, analyzing, documenting, validating, and managing the needs of users for a software system. It starts with **Requirement Elicitation**, where requirements are collected from stakeholders through interviews, surveys, observation, and discussions. Next is **Requirement Analysis**, where the collected requirements are studied to remove conflicts, duplication, and ambiguity and to check their feasibility. In **Requirement Specification**, the finalized requirements are clearly documented in a **Software Requirement Specification (SRS)**, including functional and non-functional requirements. **Requirement Validation** ensures that the documented requirements are correct, complete, consistent, and meet the users' actual needs. Finally, **Requirement Management** involves tracking, controlling, and updating requirements whenever changes occur during the software development process.

Conclusion:

The experiment successfully demonstrated the preparation of a Software Requirements Specification for the RainMumbai Index prediction system, covering functional requirements, interfaces, system attributes, database requirements, security, performance, maintainability, and Machine Learning functionalities.

Grade: AA / AB / BB / BC / CC / CD /DD

Signature of faculty in-charge with date

References:

Books:

1. Roger S. Pressman, Software Engineering: A practitioners Approach, 7th Edition, McGraw Hill, 2010.
2. Technical report on Guidelines for Documents Produced by Student Projects In Software Engineering based on IEEE standards
3. https://www.sharelatex.com/